Validation of knowledge-based systems: a reassessment of the field

Feras A. Batarseh & Avelino J. Gonzalez

Artificial Intelligence Review
An International Science and Engineering Journal

ISSN 0269-2821

Artif Intell Rev
DOI 10.1007/s10462-013-9396-9



Springer

Your article is protected by copyright and all rights are held exclusively by Springer Science +Business Media Dordrecht. This e-offprint is for personal use only and shall not be self-archived in electronic repositories. If you wish to self-archive your work, please use the accepted author's version for posting to your own website or your institution's repository. You may further deposit the accepted author's version on a funder's repository at a funder's request, provided it is not made publicly available until 12 months after publication.



Author's personal copy

Artif Intell Rev
DOI 10.1007/s10462-013-9396-9

Validation of knowledge-based systems: a reassessment of the field

Feras A. Batarseh · Avelino J. Gonzalez

© Springer Science+Business Media Dordrecht 2013

Abstract The subject of validation and verification (V&V) of knowledge-based systems (KBS) has been one of decreasing importance in the last decade. Research and development in the field reduced drastically. One of the main reasons is the persistent software challenges and failures. These failures have been categorized in different ways. One initiative however, which most researchers in the field agree upon, is that the only way to eliminate these problems is by rigorously performing V&V. Although there have been vast improvements in the field of V&V methodology, studies indicate that KBS industry still lacks rigorous validation methods. In this paper, we review the most important validation paradigms described in literature for KBS during the years of their fame. Additionally, this article studies the significant methods, aims to reassess these methods in light of recent advances, and propose new future directions for validation of KBS.

Keywords Knowledge-based systems · Validation · Lifecycle model · Software testing · Test cases

1 Introduction

Researchers in KBS have traditionally focused more on operational and representational aspects of KBS. This was likely because those problems arose earlier and were crucial to building the knowledge-based systems in the first place. As these issues were successfully addressed and full systems became available for commercial use, validation and verification became more important. This led to a flurry of research on validation (and verification) in the 1980s and early 1990s. Nevertheless, the issue of validation has not yet been fully resolved. More work has been reported on the automated verification of knowledge-based systems, for which general approaches may have been developed, than on their validation.

F. A. Batarseh (✉) · A. J. Gonzalez
Intelligent Systems Laboratory (ISL), Department of Electrical Engineering and Computer Science, University of Central Florida, 4000 Central Florida Blvd., Orlando, FL 32816, USA
e-mail: feras_batarseh@yahoo.com

Published online: 31 January 2013

Author's personal copy

Still however, much has been written in the technical literature about knowledge-based system (KBS) validation over the last three decades. Much of this has revolved around one or more *approaches* to validation. Such approaches have been in the KBS validation literature since its early days and include general concepts such as *validation through simulation* (Slafer 2001), *validation through analysis of heuristics* (Lee and O'Keefe 1994), *face validation*, *predictive validation*, *formal methods*, *test case validation*, *validation through graphical representation*, *sensitivity analysis*, *field testing* (Batarseh 2011), *Turing testing* (Turing 1950) as well as several other less meaningful ones. With the exception of test case validation, these approaches only provide a general description of how validation could be done. Because of their general nature, however, these approaches have not directly resulted in practical methods for validation of real-world knowledge-based systems. These approaches cannot by themselves be used to ensure the validity of a specific knowledge-based system. We do not describe most of these approaches in this paper, as there is little that is new there. However, three approaches merit further description, because they provide a base for multiple methods we present in this review.

The first exception to the above that can potentially break out into a practical way to validate KBSs is Context-Based Testing (CBT). CBT takes into account the contexts in which the system will be used in the real world when performing validation. It is considered a form of Agile testing (Mukkavilli 2011). CBT urges testers to modify their testing practices based on a number of parameters, such as the desires of stakeholders; more about this later in this paper. The second approach is the Turing test approach. It is one of the most acquainted approaches in AI. The label Turing Test, of course, comes from Alan Turing's seminal test for intelligence. In validation of KBS, the Turing test compares the results of executing the system to that of what the expert anticipates. We will discuss methods based on Turing test later in this paper. The third (and most important) approach is case testing. Case testing is the use of test cases to execute multiple possible processes/paths of a KBS. It is the most common validation approach in industry. Test cases can be generated randomly, without thought to exactly what is being tested. That is referred to as informal case testing. On the other hand, a plan to derive test cases in a more rigorous manner to ensure coverage of the knowledge base is referred to as "formal" case testing. The design of the test cases is often a key and essential step in this approach. More about test cases is discussed later.

What we refer to as validation *methods*, on the other hand, could be directly applied to ensure a valid system because they provide a specific procedure to follow when validating knowledge-based systems. These methods introduce a step-by-step approach towards system validation. Such validation methods have likewise been described extensively in the literature and are reviewed and discussed here.

Validation methods have been categorized in the past into two categories according to the level of abstraction of the testing process. *Black box testing* validates the system by testing inputs and outputs only, without regard to the intervening details. *White box testing* is the opposite of black box testing and scrutinizes the makeup of the system in a very detailed fashion (Sommerville 2007). Preece (2001) categorizes methods differently in his evaluation study on knowledge-based system validation and verification. The most common category that he defines is that of *inspection*. This type of validation method involves an informal manual test of the knowledge base. The expert merely looks at the rules/cases and their relations in the system and informally evaluates them based on his/her expertise. As one would expect, validation by inspection is also the least reliable. *Manual checks* and *face validation* are approaches that can fall into this inspection method category. They are performed by the knowledge engineers after deployment of the system, and most of the times with user involvement.

Author's personal copy

However, in this paper, we categorize validation methods into two, more meaningful categories: (1) Methods that validate the knowledge base, and (2) Methods that validate the entire knowledge-based system. Methods that fall into the first category (knowledge base validation) tend to be white box methods, while those belonging to the second category are more likely to be of the black box type.

This paper is organized as follows: Section 2 reviews validation methods for KBS in the two categories mentioned above. Section 3 discusses incremental validation methods, especially those linked to lifecycle development models. Finally, Section 4 presents an overall discussion of the validation methods presented, discusses validation methods relation with management and maintenance and introduces possible future avenues for research.

2 General KBS validation methods

This section describes and discusses several methods used to validate the KBS once development is completed. We begin with those methods that seek to validate the knowledge base only.

2.1 Knowledge validation methods

Gonzalez and Barr (2000) define validation as the process that assures that the output of the KBS matches the output of human experts for the same set of input data. In knowledge-based systems, the most important part is the knowledge base, as that is where the human knowledge is represented. Therefore, it is most important to thoroughly validate the knowledge base. Numerous methods (Min et al. 2006; Zlatareva 1992; Kelbasa 2004; Mengshoel and Delab 1993) have been introduced to validate the knowledge base. One of the first ideas conceived was *overgeneralization/overspecialization*. This concept was first introduced in SEEK and SEEK2 (Ginsberg et al. 1985), one of the first automated validation methods published in the literature. Overgeneralization of the knowledge base is when more rules need to be added to the knowledge to make it valid. Overspecialization of the knowledge base is when rules must be removed from the knowledge base because they are redundant, wrong or not representative of the user specifications and requirements. The idea of overspecialization reduces the size of the code and the knowledge base without affecting its performance.

SEEK is considered one of the pioneers in validation of knowledge representation. SEEK was completed in 1982 while SEEK2 was completed in 1988 (Ginsberg et al. 1985).The main goal of the two methods was rule validation and refinement for knowledge-based systems concerned with diagnosis of diseases. The main reason for the development of these tools was to ensure that the knowledge-based system's conclusions were similar to known diagnostic results. SEEK provided an evaluation of the performance of the KBS and SEEK2 extended the ability of SEEK to refine the KBS. SEEK2 had an "automatic pilot" that could perform validation tasks without the involvement of humans. Additionally, SEEK2 covered a wider variety of KBS domains than did SEEK, as it used a meta-language that represented the system in metadata. SEEK and SEEK2 have been widely used to validate many KBS.

Another idea for achieving a valid system is to keep track of *knowledge documentation* as a reference for measuring the validity of the knowledge base. The knowledge documentation includes knowledge extracted from the expert during analysis phase represented in different forms (such as tables and diagrams). Comparing the system knowledge base against its knowledge source provides a natural, albeit informal means to validate the system.

Author's personal copy

One validation method based on this concept of comparison to knowledge documentation is KVAT (Knowledge Validation Tool) (Mengshoel and Delab 1993). KVAT performs knowledge elicitation and validation for frame-based and logic-based systems. KVAT's knowledge elicitation is based on labeled grids-indexed tables that represent the knowledge obtained through interviews with the expert and case histories. The KVAT validation method starts by performing manual case testing on the system. If KVAT finds any error during case testing, it locates the source of the inconsistency in the knowledge and compares it to the elicitation documents to perform refinement on the knowledge. This method is designed for use across all development phases, but it is mainly useful for validation of the knowledge representation. KVAT automates the stage that compares the knowledge to the validation documents, but case testing is still manual and informal.

Formal methods provide a well-defined process for testing the system. Santos and Dinh (2004) presented a formal validation method using case testing and probabilistic Bayesian networks. To use their method, the knowledge base should be in a *Bayesian knowledge base* (BKB)format-Bayesian networks represented as *if-then* rules. This is an advantage because knowledge engineers are typically familiar with such a conditional paradigm. For validation, a set of test cases is created by deriving it from the Bayesian knowledge base and the initial user requirements (Santos and Dinh 2004). Every test case is a pair composed of *evidence* and *expected answer*. The evidence represents the requirement location in the knowledge base, and the expected answer is the correct solution. These pairs (evidence and expected answer) change depending on the incoming results and outputs of the testing process. Changing the set of test cases means that new test cases will be added to or deleted from the set. In this method, every requirement from the users is represented by a test case. Furthermore, this method represents the mentioned *evidences* and *expected answers* as nodes in a graph, which is itself validated by checking the links between the nodes.

A more generalized validation method for knowledge representation was introduced by Zlatareva (1992). Her method is called *Validation, Verification and Refinement* (VVR) and its title defines its function well. For validation, VVR uses the concepts of positive and negative test cases to modify and refine the knowledge base. Validation starts by manually generating test cases for the system being validated. Positive test cases are used to show the validity of part of the knowledge base. They are represented as three variables: *input*, *expected result* and *actual result* (Zlatareva 1992). Negative test cases are used to show that the negation of the inference outcome is wrong. The idea of positive and negative test cases unveils different kinds of problems in the system being validated. In some cases, during validation and refinement, adding more rules to the knowledge base could be the solution. In such cases, however, the knowledge engineers would not know whether redundancies are added to the knowledge base or to the intelligent program's code because of this. This can make the size of the program grow without justification. This would lead to the consistent need for regression testing. Regression testing has to be performed after changes are done on the software; and in some cases could be complicated and time consuming.

Another method for knowledge validation is based on graphical representation using conceptual graphs. It was recently presented by Dibie-Barthelemy et al. (2006). They introduced a method to detect anomalies in the knowledge base. They call the process validation, although detecting anomalies is traditionally more a part of the verification process according to many definitions (Tadji and Laroussi 2009; Morell 1988; Marcos et al. 2000). Nevertheless, in this method, the knowledge is separated into two types, *terminological* (contains the main knowledge and the basic terms) and *assertional* knowledge (contains a number of conceptual graphs that are based on the terminological knowledge). Terminological knowledge helps in forming the assertional knowledge to be validated manually by the experts. Graphical

Author's personal copy

representation is helpful in envisioning the knowledge base, but without a tool that can transform the knowledge to graphical form, it can be a difficult and time-consuming method.

Another notable tool presented in the literature is called EITHER (Zlatareva and Preece 1994). This system is a method embedded in a tool that uses the logic of overgeneralization and overspecialization discussed earlier. Furthermore, EITHER uses a *theory tree*, which is a binary tree with AND/OR representations. If the error is caused by overspecialization, the rules associated with the problem will be removed after searching for replacements from the tree. EITHER only validates the knowledge base; it does not validate the entire system. Next section introduces validation methods that execute black box testing on the entire system.

2.2 System validation methods

The methods reviewed in this section compare the system's output to that of the expert and are concerned about more than merely the contents of the knowledge base.

Min et al. (2006) propose a method to perform system validation by using validation software agents, or in other words, knowledge-based systems that can assist in the validation of other knowledge-based systems. This validation method is founded on using three knowledge bases for validation: (1) the *validation knowledge base* (VKB), which represents the expected behavior of the system; (2) the *decision knowledge base* (DKB), which has the inference knowledge for selecting a validation method for a given system; and (3) *validation techniques base* (VTB), a KBS that includes all the validation methods. This validation method starts by analyzing the target knowledge-based system. A validation method is selected from the VTB by the DKB according to the VKB system's representation (Min et al. 2006). Another module in this method is the *evaluator*. As the VKB represents the expected final behavior, the evaluator compares the results of the validation technique selected from the VTB to VKB's representation. DKB selects a validation method based on the representation of VKB using a predefined set of rules from the expert. The evaluator creates a report as a result, which consists of both the validation outcome and the VKB representations. Different parts of the validation system and how they are interrelated are shown in Fig. 1 (Min et al. 2006). As the figure illustrates, three expert systems and the evaluator are the main parts of this validation method. This method relies on the correctness of the decision-making ability of the DKB, and the representation logic of VKB. It is also based on the assumption that the three expert systems (VKB, VTB and DKB) have been validated and verified a priori, thus posing the chicken-and-egg dilemma. Different validation methods could be incorporated within this validation system (specifically into VTB), which is an advantage. This system can be updated periodically to include new validation techniques. However, the authors did not validate their software agent in their paper. This is a problem that applies to any validation software tool—how does one validate the validation tool?

Another overall system validation method called VESA was introduced by Knauf et al. (2007). They also introduce a validation knowledge base (VKB), but it is different in concept from that introduced by Min et al. (2006). The VKB of Knauf et al. is the collection of the expertise of many experts as well as of the validation expert system agent (VESA). VESA is a software agent that represents one expert's knowledge. This method uses the test case validation approach. The VESA and VKB modules can theoretically replace the validating experts after a certain amount of time for learning to validate knowledge-based systems. However, that was not explicitly tested in their paper.

VKB is a group of test cases and a set of results that represent the correct results of each test case (Knauf et al. 2007). VKB consists of knowledge dealing with: test case generation,

Author's personal copy

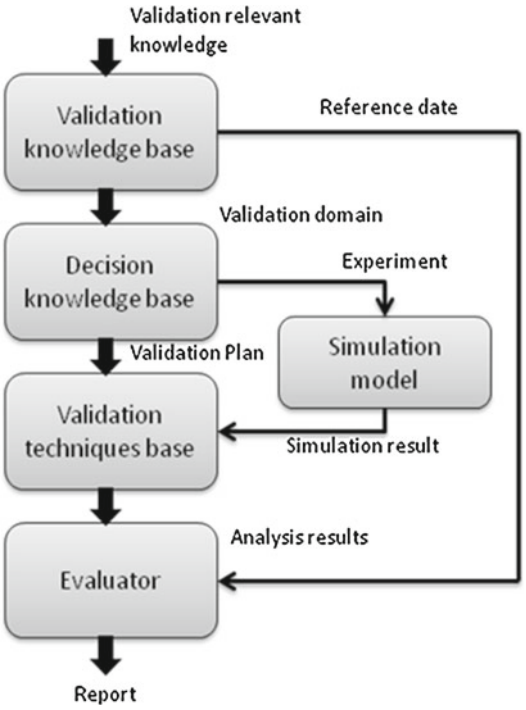


Fig. 1 VKB validation method (Min et al. 2006)

test case reduction using ReST (a "reasonable set of test cases", as defined by the authors), test case rating, evaluation and refinement. The main idea in this method is that when manual testing starts, VESA could replace any of experts involved in the testing if that expert must be absent from the validation process. If the expert is not available during testing, the VESA should have gathered enough knowledge from the expert and VKB during earlier stages of system development so it can replace him/her in the testing process. VESA uses VKB for decision-making. Furthermore, VESA saves the experts' test cases ratings and certainties that can be used when replacing the expert. This method can reduce human involvement in the validation process (Knauf et al. 2007), which of course, is a desirable practice. The VESA method presented is successful in involving software agents, decreasing time & labor costs, plus increasing reliability.

Another validation method based on test cases was introduced by Smith and Kandel (1900). They presented their method through a tool called the CASE VALIDATOR. It's an automatic record-keeping tool of the test cases used for validation and it provides analysis of these test cases and how they cover the knowledge base. The test cases are manually entered into the tool through a user-friendly interface. For every test case, the results, certainties and conclusions are saved after the execution. Part of the tool is called the *record-keeping facility*. It keeps track of any changes to the program and compares the results of the test cases associated with that change. The *coverage analyzer* is another part of this method/tool. It consistently checks for any part of the program not covered by the test cases in the record-keeping facility. CASE VALIDATOR, however, is a manual approach that is impractical for large knowledge bases (Gonzalez and Barr 2000; O'Keefe et al. 1987; Min et al. 2007).

A modified Turing Test was used to validate the pioneering MYCIN system (Vinze et al. 1991). In MYCIN's Turing Test validation approach, several physicians of different levels

Author's personal copy

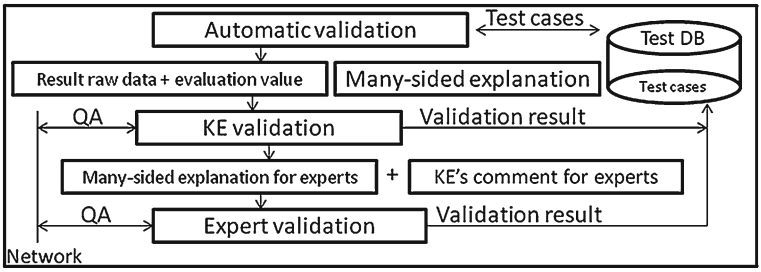


Fig. 2 Bi-directional many-sided explanation typed multi-step validation method (Onoyama and Tsuruta 2000)

of expertise, from medical students to attending physicians to senior medical faculty solved several cases taken from actual clinical experience. Their results were provided anonymously to a panel of "validators" composed of senior medical faculty who evaluated every test case and result proposed by the first group of physicians and medical students to assess their correctness. One could argue that the test cases were in fact an interrogation of several human experts plus the MYCIN system for their answers vis-à-vis the test cases. The second panel of evaluators was in effect, (indirectly) determining which of the interrogations were by the human and which was by MYCIN. In any case, the MYCIN developers concluded from this Turing Test that MYCIN actually made better decisions that even the most senior of medical faculty members. While this cannot be considered an actual validation of MYCIN, the modified Turing test they conceived has been considered highly suitable for validation of KBS. This was the inspiration taken by Knauf et al. (2002) in their Turing Test approach.

Onoyama and Tsuruta (2000) present a validation method to address the intense human effort required in the Turing test; the authors presented a method called the *Bi-directional, many-sided explanation, typed multi-step validation* method. It is worth mentioning here that when experts are exposed to tests, their knowledge will be under scrutiny, as all their knowledge will be in the system. Onoyama and Tsuruta (2000) assert that some experts might think that they will lose their inherent superiority as experts, which might affect their inputs into the test. This is considered a drawback for the Turing Test approach. Onoyama et al., therefore, sought to eliminate this factor. This method is a multiple step validation process. The tests that the expert performs are only those that neither the computer nor the knowledge engineer can perform. The rest are left for the knowledge engineer and the automated computer process. The computer stores the answers generated by the system, the expert and the validation experts during all the tests. Comments are also added to the system by the knowledge engineer and the original (those involved in system development) as well as the validating experts. The results of the validation are stored in the database as a validation case base, which could be used in subsequent validation exercises. After the validation process, all the results from the expert, knowledge engineer and system are aggregated and documented. The Bi-directional-many-sided-explanation-typed-multi-step validation method is illustrated in Fig. 2 (Onoyama and Tsuruta 2000). This method has two major drawbacks. The knowledge engineer and the expert perform validation separately, which can lead to conflicting results. Additionally, this method requires effort in project management and communication between experts and knowledge engineers that can also be expensive.

Another validation method was introduced through a software tool KJ3 by Wu and Lee (2002), which stands for knowledge judgment version 3.KJ3 uses a general approach for

Author's personal copy

graphical representation. KJ3 consists of four main modules: the *user interface* (facilitates the usage of the tool), the *validation module* (the main module responsible for validating the system), *the axiom translator* module (converts the validation tasks into reach-ability first order logic) and the *proof interpreter* (provides messages to the user about the validation results). In this tool, high-level Petri nets are used to represent the knowledge base. These Petri nets are a meta-representation scheme to present the knowledge. To validate the Petri net means validating the expert's knowledge. It is required that such "equivalence is in a one-to-one mapping relationship" (Wu and Lee 2002) between the knowledge and the Petri net nodes. The selected procedure has to mirror the inference practice of the original knowledge base. KJ3 could be used to validate any knowledge representation.

A validation method that aims to reduce a large number of test cases was presented by Gonzalez et al. (1996). In their method, they based test cases selection on heuristic techniques. In their paper, they briefly describe and validate the *Generator Artificial Intelligence Diagnostics* (GenAID) system (Gonzalez et al. 1996). GenAID was a knowledge-based system developed in the 1980s at Westinghouse Electric Corp to diagnose faults with large electrical generators. Because the cost of system misdiagnoses can be high, customers sought a rigorous assurance that the GenAID system was in fact reliable. GenAID was developed as a group of semi-independent modules. The paper described two types of modules, *self-contained* modules and *overlapping* modules. Overlapping modules are harder to validate, as they cover more than one module in the system. The knowledge base is represented as a graph (one for each module) and the test case set is generated from these graphs using heuristics. Test cases are created for every path in the graph. However, depending on the heuristics, some paths demand that several test cases be created because of their importance. The heuristics, therefore, were used to vary the intensity of the path testing (with different values for inputs). One disadvantage of this method is that heuristics are fallible and they do not ensure a 100 % valid system.

Abel et al. (1996) had a different take on this issue; they decided to reduce the number of test cases needed to validate a knowledge-based system. The idea of testing every input to the system is impossible to implement in most cases. Abel et al. (1996) used several criteria to reduce the number of test cases necessary to achieve successful validation. Their method seeks to build a minimized test cases set that is sufficient to warrant a valid system if all test cases are solved correctly by the system. They begin by making the logical assertion that a KBS can be said to be valid if it correctly solves an*exhaustive set of test cases*. However, for all but the most trivial of systems, this would include an astronomically large number of test cases. So, they compute and build a *quasi-exhaustive set of test cases* (QEST) instead. This set of test cases is represents all the system cases in a reduced number of test cases that is functionally equivalent to the full exhaustive set. In the *QEST*, it is assumed that if a specifically selected subset of the test cases (T) is valid, then the whole set (S) can be said to be valid. Test cases sets are defined and executed for each set a subset of test cases (T). A set of mathematical classifiers is used to categorize the knowledge into sets where an object can belong to one or more sets. In this method, the authors used statistical means to achieve a valid system. A year later, Abel and Gonzalez (1997) criticized the *quasi-exhaustive set of test cases* as still being unacceptably large. They introduced another method to further reduce the number of test cases for validation. In their second method, a criteria-driven reduction of test cases is used to generate a *reasonable set of test case* (ReST).

Abel et al. also propose a means of validating the system once the set of test cases has been minimized. This method includes four stages as illustrated in Fig. 3.

During the test case evaluation stage (refer to Fig. 3), the results of all the test cases are evaluated by the expert and the knowledge engineer. These evaluations are analyzed in the

Author's personal copy

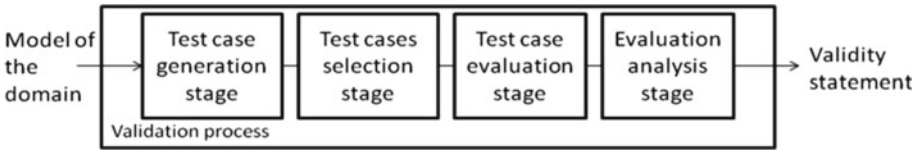


Fig. 3 Validation process proposed by Abel et al. (1996)

fourth step. With this analysis, a conclusion is reached on the level of validity of the system. During the test case selection stage, a criterion is defined by answering the question of how well should the model perform for it to be considered valid. Answering this question requires looking at different criteria: domain related, user related, and expert related. This method ensures the reduction of the set of test cases.

Mosquiera-Rey and Moret-Bonillo (2000) present a tool called SHIVA (Spanish language acronym for Heuristic Integrated Validation System). SHIVA uses a black box general validation approach that contains three main steps: planning, application and implementation. In the first step, planning extracts the domain, user and system's characteristics. During the application step, SHIVA aims to decide on the quantitative method to be used for validation and applies it to the system. SHIVA selects one of the following quantitative methods for validation: *pairs' method*: Two or more experts' interpretations are compared to the interpretations of the system, *association measurement*: models the outcome of the system and the expert then calculates the degree of linear association between them and *agreement measurement*: for a given set of rules or any other knowledge representation. An agreement index is defined, and the knowledge is validated in comparison to this index. The output of this process defines the distance between the index and the knowledge under test (Mosquiera-Rey and Moret-Bonillo 2000). SHIVA is not incorporated into the development process, which is a drawback because the knowledge engineer needs to pick a validation method for the system.

Zlatareva (1998) argued that validation and development should be fully integrated. They therefore introduced a validation tool called DIVER, which is incorporated into the spiral development model. During each validation cycle, DIVER finds potential logical and semantic errors defined in the knowledge base. Detected errors are sorted into levels. This sorting is based on which inconsistency happened before; earlier ones need to be solved first because inconsistencies at earlier levels may have a negative effect on later ones. When the development using this method is over, there is no need for validation except for the last modifications on the system. Complete system testing is recommended, however.

A generic validation tool called CORUS was introduced by Abdallah et al. (2007). It is built on top of another system called the *resolver* (Abdallah et al. 2007), which is used to handle numerous decision criteria. Resolver is a software system that helps in taking decisions regarding which validation approaches to use, based on inputs such as, size of system and type of system. CORUS was developed following the spiral model and has the following parts: a database, knowledge base, inference engine, a case acquisition tool, a user interface and an explanation module. The system validates the results based on assigning *profile*'. For each of the returned solutions, the system associates a profile so that equal solutions have equal profiles. The expert and the knowledge engineer manually compare the outcome of the system to the recommended outcome from the tool.

Another generic tool called KRFOCL (knowledge rules fold cross validation) was introduced by Murrel and Plant (1997) based on backward chaining. This tool automates the tracing of errors back to the rule base and selects the rule or set of rules that are responsible

Author's personal copy

for the errors. Traceability is used to link the outside view of the system to its inside. KRFOCL can detect missing, contradictory and unnecessary rules. This tool requires a large set of test cases to initiate the process of traceability and the test cases should run to indicate where the errors are happening. It would be useful to have traceability back to the requirements, so the real world would be represented in a more complete manner in the system.

Incomplete knowledge bases do not fully reflect the knowledge of the expert, and incorrect knowledge bases represent their knowledge incorrectly. Considering this, Lockwood and Chen (1995) proposed that knowledge validation and knowledge elicitation should be performed in parallel. They introduced a tool called EMBODY (Expert Modeling Based on Decision Modularity) that performs both elicitation and validation. EMBODY can generate the rules of the knowledge base, using *knowledge charts* to generate the rules. Knowledge charts act like a bridge between the expert and the rules. They are intended to capture the 'flow of thought' of the expert and illustrate it. EMBODY validates as it goes, and when the last step is reached, only the last step needs to be validated. As claimed, EMBODY is embedded into the development process, but it does not validate against the real world, which is a drawback, because that is the essence of validation. EMBODY lacks a link to the user requirements and the domain. It captures the expert's knowledge in a valid way, but not the intended performance of the system. EMBODY is similar to incremental validation methods, but it is not a complete one. In order to emphasis on the importance of incremental validation, we decided to dedicate a section to it. Incremental validation methods are discussed in the next section.

3 Incremental validation methods and discussions

It should be noted that many efforts exist towards making validation a more robust process. Nevertheless, it is important to note here that validation in the conventional software world is more developed and mature than it is in knowledge-based systems. Conventional software validation methods cannot be directly used for knowledge-based systems, however, because of the differences in the nature of the two types of systems. The most important of these differences are:

1. Knowledge based systems generally use off-the-shelf inference engines that normally do not require validation; it is (possibly incorrectly) assumed that they are already valid.
2. Although conventional software systems have knowledge embedded into them, they do not use human knowledge explicitly the way KBS do.
3. The process of building a knowledge-based system is different than that of conventional software systems. KBS have a different lifecycle because of this difference in their nature.

Researchers have studied incorporating validation into a conceptual software development model (Lee and O'Keefe 1994; Batarseh 2011; Turing 1950) but there has been limited success. After working with different general validation approaches, Lee and O'Keefe (1994) concluded that "We should build validation into the development cycle". However, a clear-cut paradigm to build KBS software is still emerging. A number of methods that purport to do this have been introduced in the literature but none is widely used or even agreed upon. To address this, researchers have tried to embed knowledge-based systems development steps into some conventional software models. We discuss some of these methods below.

Lee and O'Keefe (1994) incorporated knowledge-based systems development phases into the spiral model. It is important to note that in the spiral model, the authors suggested using two types of validation approaches, case testing and field testing through data collection.

Author's personal copy

This suggests the need for more than one validation approach for the outcome of validation to be sufficient. This also indicates that validation is better when performed incrementally (spiral iterations) to avoid late risks and unfixable problems.

Researchers have likewise looked into modifying conventional software validation models for use on knowledge-based systems (Lee and O'Keefe 1994). Vermesan and Hogberg (1999) developed a system and a tool that evaluates software validation methods and checks their applicability to knowledge-based systems. They introduced a scale of categories of classifications, including HA (highly applicable), A (applicable), LA (Low applicability), and NA (not applicable). These classifications help in determining whether a method is applicable or not based on *mutation testing. Mutation testing* is defined as the process of modifying program's source code in a repetitive manner, and in some ways is similar to Genetic Programming. These so-called *mutations* are based on well-defined *mutation operators* that either imitate user errors (for example, using the wrong operator) or define valuable test cases. The purpose is to help the tester develop effective tests, or locate weaknesses in the test data used for the program or in sections of the code that are seldom or never accessed during execution. These approaches are control flow analysis and cause-effect graphing. For most of the techniques developed for validating conventional software, applying these to KBS is not straightforward. The framework that Vermesan and Hogberg (1999) presented accepts a validation method as an input and produces a method oriented to knowledge-based systems as an output, with some indication as to how similar are they to each other.

Other researchers have looked at the problem from different points of view. Instead of deriving validation methods from other fields and defining a software development life-cycle model, they developed a validation method that is life-cycle independent. Wells (1993) defined such a life-cycle independent method called VIVA. VIVA consists of two parts: a set of productions that describe the system, and a set of steps to show how the products can be used to validate the system. The VIVA method views the system as a group of entities and a group of links between them. The links have the following properties: existence (reflects whether the link between two objects exists or not), connection (the entities that the link connects), referencing (whether entities at each end reference themselves or each other in a correct manner), completeness (describes if the representation is complete), and correctness (describes if the link is connecting two right entities).Its authors are still defining VIVA and will continue doing so by trying to align VIVA to different kinds of software systems in any domain.

Batarseh et al. (2010) presents a method called MAVERICK (Method for semi-Automated Validation Embedded into the Reusable and Incremental Common KADS). This method is presented in Fig. 4. As the name implies, MAVERICK is embedded into CommonKADS and is a semi-automated and incremental method. CommonKADS is a lifecycle model used to develop knowledge-based systems. CommonKADS offers a *de facto* standard for building knowledge-based systems. Additionally, CommonKADS is a knowledge representation-independent lifecycle model. It has powerful models that can be used to represent many domains.

MAVERICK is based on case testing and selecting an appropriate set of test cases to validate the system. It makes use of results of prior test cases in an incremental validation procedure. MAVERICK uses results from previous validation iterations to select the next set of test cases. This facilitates defining a minimal set of test cases that provides complete and effective system coverage. MAVERICK was tested and the experiments wielded a success rate of 94 % (Batarseh et al. 2010).

Author's personal copy

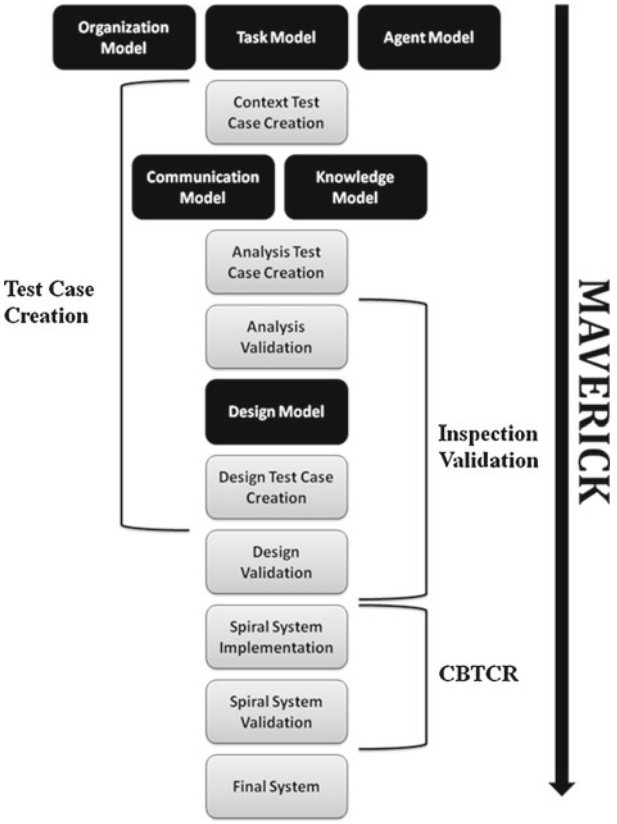


Fig. 4 MAVERICK validation method (Batarseh et al. 2010)

4 Validation discussions and conclusions

In summary, methods for the validation of the knowledge base were reviewed such as BKB, VKB, KVAT, SEEK and SEEK2. Furthermore, methods for validation of the entire knowledge-based system were also reviewed, such as Bi-directional many-sided explanation typed multi-step validation, VESA, CORUS, Decision KB, CASE VALIDATOR, KJ3, VVR and quasi-exhaustive set validation. Additionally, other multi-purpose validation tools were developed such as SHIVA, DIVER, EITHER, CORUS and EMBODY. Some of the defined models were confirmed by the authors to be a success through experimentation (SHIVA, KVAT, KJ3, SEEK, SEEK2, VESA, CORUS, MAVERICK), while others failed to provide such testing (BKB, VKB, Decision KB, CASE VALIDATOR, VVR), while yet others were not tested or evaluated (EMBODY, DIVER, EITHER). We continue our discussion by revisiting the concept of validating within a lifecycle model.

4.1 Validation within a lifecycle model

As presented in this paper, there are many validation approaches for knowledge-based systems, but how do we decide which one to use? Some of the validation approaches are suitable for use with certain types of systems in certain domains (e.g., validation methods for

Author's personal copy

real time systems).Different validation approaches could be used at different phases of the knowledge-based life cycle. For example, field testing might be used after the implementation phase is complete, while graph validation might be used after knowledge elicitation.

As presented in the previous sections, validation should be performed at any and all levels of the system development stages. There are different methods for validation that could be used for different purposes. Some of the validation methods presented performs validation on the knowledge base. Other validation methods perform validation on the entire system. Although it was discussed by O'Keefe et al. (1987), none of the early validation methods reported in the literature proposed performing validation across all phases of development. A number of methods have been introduced in the literature but none is widely used or agreed upon. A clear-cut paradigm to build software is still emerging.

One of the obvious advantages of what O'Keefe et al. (1987) mentioned is the issue of Heisen and Bohr bugs. Bohr bugs (named after the Bohr atom model) are those bugs that could be easily regenerated for testing purposes. However, Heisen bugs (named after the Heisenberg uncertainly principle) are the ones that are difficult to regenerate or duplicate. Obviously, Bohr bugs are easier to trace. Thus, a well-defined life-cycle model creates more Bohr bugs than Heisen bugs, which means less validation and maintenance time (Cook 2010).

Knowledge-based system development is widely known to be a multi-phase process. In every step, major changes are potentially made to the system. Therefore, multiple modules would need to be validated. Selecting a validation method for every module or for every step is a troublesome process because many reliability, regression and dependability problems might appear. Many researchers (Lee and O'Keefe 1994; O'Keefe et al. 1987; Onoyama and Tsuruta 2000; Batarseh et al. 2010) have agreed that incorporating the validation process into a software development life cycle is vital. Another aspect of KBS validation is that they need much effort from the knowledge engineer, and in most cases, knowledge engineers would rather use simpler validation methods. Furthermore, measuring the validity of a system is still a challenging process. It is still difficult to know when to stop performing validation. How will the knowledge engineer know when the system is valid and avoid immature validation conclusions? This immaturity could lead to software failures, financial loss and in some cases injury or loss of human life. Some of the methods described above (i.e., MAVERICK Batarseh et al. 2010) successfully addressed the issue of fully incorporating validation in all development phases.

Another costly aspect is maintenance, which is performed after testing, validation and verification, and requires a significant effort/cost. Traceability is the main difficulty when performing maintenance on a software system. It is evident that to properly maintain a knowledge-based system, the requirements need to be well defined and that they could be traced all the way down to the implementation level. In maintenance, most of the burden is on the expert and the knowledge engineer and user involvement is typically lacking. While this is true for most late phases in software development, it is not the case for the early requirements and knowledge extraction phase. Involving the user in validation and verification is not yet fully achieved in this research field.

Automating the validation process is another important issue. The definitive step in automating testing is to perform automated refinement. Automated refinement would lead to a full autonomous testing process. This would not only save time and effort but would insure that no human error will occur because of manual execution or refinement of tests. This and other suggested future directions are introduced in the next sub-section.

Author's personal copy

4.2 Future directions

Knowledge-based systems lack automated validation processes and systematic refinement of different parts of the system, which is generally not the case for conventional software systems. For the most part, KBS still lag behind with regards to validation when compared to conventional software. Therefore, we have the following recommendations for future directions in this field:

1. Unify validation and verification instead of performing them separately. This would present comprehensive means for developers, quicken the process of development and most importantly make it less complicated to achieve a valid and verified system.
2. Learn lessons from the conventional software testing research community as it has advanced much more than that for KBS. Studies mentioned previously, such as Vermesan and Hogberg (1999) have looked into this issue, but because empirical software engineering have come a long way, it is important to consider its findings and adapt them to KBS. (Example: apply more agile processes to KBS.)
3. Define the different advantages and disadvantages of different lifecycle methods. Different lifecycles have been defined for KBS, however, no guidelines are specified on which model is better for any given case. It is important to categorize KBS development approaches within those lifecycle models.
4. In KBS, the knowledge engineers tend to involve the expert more and typically reduces user involvement. Involving the user in validation and verification would be very beneficial to the process. Additionally, as previously discussed, performing KBS development within a lifecycle model (such as CommonKADS) is crucial to the success of validation.
5. Automate the V&V process, but more importantly, automate the refinement process. Manual processes are not preferred anymore, and any V&V refinement process should be fully automated and easy to implement on the system. These recommendations seek to improve the development and V&V processes of KBS altogether.

Different validation approaches produce different loads on the experts and the development team. Experts cannot perform classical validation by only examining inputs and outputs for the system. Such validation burdens the expert and the knowledge engineer. It is important to follow a defined process that is designed to increase system coverage. In a software development environment, managers take great interest in the final results of testing and testing coverage levels. Using a lifecycle model is the most agreed upon way to achieve measurable parameters to help managers in planning. From a management point of view, planning and scheduling are essential to the software development process. Therefore, validation methods should be tailored to help management follow the process and evaluate the performance against deadlines. Preferably, validation should be performed in phases where it is clear where the phase starts and where it ends. This would help in planning and setting deadlines. Management requires a solid and explicit organization for software development projects.

References

Abdallah K, Mohammad T, Louis F (2007) Validation of intelligent systems: a critical study and a tool, Corus. In: Proceedings of the, international journal of soft computing, pp 191–198

Abel T, Gonzalez AJ (1997) Utilizing criteria to reduce a set of test cases for expert system validation. In: Proceedings of the 10th FLAIRS conference 1997, pp 402–406

Author's personal copy

Abel T, Knauf R, Gonzalez AJ (1996) Generation of a minimal set of test cases that is functionally equivalent to an exhaustive set, for use in knowledge-based system validation. In: Proceedings of the 9th FLAIRS conference 1996, pp 280–284

Batarseh F (2011) Incremental lifecycle validation of knowledge-based systems through commonKADS, Ph.D. Dissertation at the University of Central Florida, Spring 2011

Batarseh F, Gonzalez AJ, Knauf R (Sep 2010) Validation of knowledge-based systems through commonKADS. In: Proceedings of the 55th Internationales Wissenschaftliches Kolloquium (IWK), Ilmenau, Germany

Cook D (2010) Software processes? How bohring? Back talk in the magazine of crosstalk. J Def Softw Eng 23(1):39

Dibie-Barthelemy J, Haemmerle O, Salvat E (2006) A semantic validation of conceptual graphs. In: Proceedings of the journal of, knowledge-based systems, pp 498–511

Ginsberg A, Weiss S, Politakis P (1985) SEEK2: a generalized approach to automatic knowledge-base refinement. In: Proceedings of international joint conference on artificial intelligence (IJCAI), pp 367–374

Gonzalez AJ, Barr V (2000) Validation and verification of intelligent systems—what are they and how are they different? In: Proceedings of the journal of experimental & theoretical artificial intelligence, October-2000, pp 407–420

Gonzalez AJ, Gupta U, Chianese R (1996) Performance evaluation of a large diagnostic expert system using a heuristic test case generator. In: Proceedings of the engineering applications for artificial intelligence, vol 9, pp 275–284

Kelbasa H (2004) Context refinement—investigating the rule refinement completeness of SEEK/SEEK 2. In: Proceedings of the 15th European conference on, artificial intelligence, pp 123–154

Knauf R, Gonzalez AJ, Abel T (2002) A framework for validation of rule-based systems. IEEE Trans Syst Man Cybern 32(3):181–196

Knauf R, Tsuruta S, Gonzalez AJ (2007) Toward reducing human involvement in validation of knowledge-based systems. In: Proceedings of the IEEE transactions on systems, man and, cybernetics, vol 37, January 2007, pp 120–131

Lee S, O'Keefe R (1994) Developing a strategy for expert system validation and verification. In: IEEE 1994, proceedings of the IEEE transactions on systems, man and cybernetics, vol 24, pp 643–655

Lockwood S, Chen Z (1995) Knowledge validation of engineering expert systems. In: Proceedings of the journal of advances in software engineering, pp 97–104

Marcos M, Del Pobil D, Moisan S (2000) Model-based verification of knowledge-based systems: a case study. In: Proceedings of the IEEE journal on software, pp 163–167

Mengshoel OJ, Delab S, (1993) Knowledge validation: principles and practice. In: Proceedings of the journal of IEEE experts systems, pp 62–68

Min F, Yang M, Wang Z (2006) An intelligent validation system of simulation model. In: Proceedings of the fifth international IEEE conference on machine learning and, cybernetics, pp 1459–1464

Min F, Ma P, Yang M (2007) A knowledge base method for the validation of military simulation. In: Proceedings of the winter simulation conference-IEEE, pp 1395–1402

Morell L (1988) Use of meta knowledge in the verification of knowledge-based systems. In: Proceedings of the 1st international conference on industrial and engineering applications of artificial intelligence and, expert systems, pp 847–857

Mosquiera-Rey E, Moret-Bonillo V (2000) Validation of intelligent systems: a critical study and a tool. In: Proceedings of the journal of, expert systems with applications, pp 1–16

Mukkavilli L (2011) Publication number: US Patent 2011/0098963 A1, Context-based testing

Murrel S, Plant RA (1997) A survey of tools for the validation and verification of knowledge-based systems: 1985–1995. In: Proceedings of the, decision support systems, pp 307–323

O'Keefe RM, Balci O, Smith EP (1987) Validating expert system performance. Proc IEEE Expert 2(4):81–90

Onoyama T, Tsuruta S (2000) Validation method for intelligent systems. In: Proceedings of the, journal of experimental and theoretical artificial intelligence, pp 461–472

Preece A (2001) Evaluating verification and validation methods in knowledge engineering. In: Roy R(ed) Micro-Level Knowledge Management, Morgan–Kaufman pp 123–145

Santos E Jr, Dinh H (2004) Consistency of test case in validation of Bayesian knowledge bases. In: Proceedings of the 16th IEEE international conference on tools with artificial intelligence (ICTAI 2004)

Slafer L (2001) (Boeing Satellite Systems) Achieving software validation through simulation. In: Proceedings of the advanced dynamics international user conference (ADIUS)

Smith S, Kandel A (1990) Validation of expert systems. In: Proceedings of the third Florida artificial intelligence research symposium (FLAIRS), pp 197–201

Sommerville I (2007) Software engineering, 8th edition, Chapter 4. published by Addison Wesley

Tadji C, Laroussi T (2009) Dynamic verification of an object-rule knowledge base using colored Petri nets. In: Proceedings of the conference of systemic, cybernetics and informatics 2009, vol 4, pp 337–352

Author's personal copy

Turing A (1950) Computing machinery and intelligence. Proc Mind LIX(236):433–460
Vermesan A, Hogberg F (1999) Applicability of conventional software verification and validation to knowledge base components: a qualitative assessment. In: Proceedings of the 5th European symposium on validation and verification of knowledge based- systems-theory, tools and practice (EUROVAV '99), pp 343–364
Vinze A, Vogel D, Nunamaker J (1991) Performance evaluation of a knowledge-based system, a validation study. In: Proceedings of the journal of information and management. Elsevier, Amsterdam, pp 225–236
Wells S (1993) The VIVA method: a life cycle independent approach to KBS validation. In: Proceedings of the IEEE AAAI conference, pp 102–106
Wu C, Lee S (2002) KJ3-a tool assisting formal validation of knowledge-based systems. In: Proceedings of the international Journal of the human–computer studies, pp 495–525
Zlatareva N (1992) A framework for knowledge-based systems verification, validation and refinement: the VVR system. In: Proceedings of the 5th FLAIRS conference 1992, pp 10–14
Zlatareva N (1998) Knowledge refinement during development and field validation of expert systems. In: Proceedings of the 11th international FLAIRS conference, 1998, pp 467–472
Zlatareva N, Preece A (1994) State of the art in automated validation of knowledge-based systems. In: Proceedings of the journal of, expert systems with applications, pp 151–168